ENEE 467: Robotics Project Laboratory
Visual Robot Perception in ROS 2
Lab 6 - Part 2 (Hardware)

Learning Objectives

This part moves perception from the simulated 2D pipeline of Part 1 onto the real robot. Working with the Intel RealSense D435i RGB-D camera, you will build a 3D point-cloud perception pipeline that detects an object on a tabletop and serves its pose, then command the UR3e arm to move its tool to that pose. The Hand-E gripper is not used in this part. The arm moves to the detected pose but does not grasp. After completing this part, you should be able to:

- Acquire and inspect a live point cloud from the RealSense D435i in ROS 2
- Downsample and crop a point cloud with a voxel-grid filter
- Segment a tabletop plane with RANSAC and cluster objects with DBSCAN
- Serve a detected object's pose through a ROS 2 action
- Move the UR3e arm tool to the served object pose with MoveIt (arm only)

Visual robot perception refers to the process through which a robot acquires, interprets, and makes sense of its environment using cameras and other image sensors. In three dimensions, the camera reports a point cloud, and the perception system must reduce that raw cloud to task-relevant structure: which points belong to the supporting surface, which points form objects, and where each object is in the robot's base frame. This part builds that pipeline stage by stage on real hardware.

Related Reading

- M. Spong et. al., "*Robot Modeling and Control*", John Wiley & Sons, Inc., 2006.
- Intel RealSense SDK: GitHub.
- ROS Perception, "*vision_opencv*" Available: vision_opencv GitHub Repository.
- M. A. Fischler and R. C. Bolles, "*Random Sample Consensus*", Comm. ACM, 1981.
- M. Ester et. al., "*A Density-Based Algorithm for Discovering Clusters (DBSCAN)*", KDD, 1996.

Lab Instructions

Follow the lab procedure closely. Some code blocks have been populated for you and some you will have to fill in yourself. Pay close attention to the lab Questions (highlighted with a box). Answer these as you proceed through the lab and include your answers in your lab writeup. There are several lab Exercises at the end of the lab manual. Complete the exercises during the lab and include your solutions in your lab writeup.

List of Questions

List of Exercises

Hardware Safety

Throughout this manual, boxes beginning with a **Question #** label contain questions you must answer as you progress through the lab:

> **Question 0.** A question that you should answer as you work through the lab manual.

⚠ **Safety and Setup Notices.** Warnings highlight critical steps for hardware safety, teach-pendant operation, and common setup pitfalls. Always read these carefully before commanding motion on the UR3e.

> ⚠ Keep clear of the robot workspace when running autonomous motion. Confirm that the pendant is in `Play` mode before proceeding.

Notes. These boxes provide background or theoretical context for the UR robot system. They are optional during lab time but useful for your report:

> UR3e kinematics are defined in a six-joint chain; MoveIt uses the URDF and joint limits to compute feasible IK solutions.

Code, commands, and output are formatted as follows:

```
ros2 launch ur_robot_driver ur_control.launch.py ...
```

```
[INFO] [launch]: UR3e driver successfully connected.
```

Files and directories appear in magenta teletype font (e.g., `~/lab7_ws/src`), while in-text code and variables use blue teletype (e.g., `object_pose`). Python type hints or XML tags appear in green (e.g., `str`, `<param>`).

1. Lab Procedure

> ⚠ Stay alert when working with the robot, and use the Emergency Stop Button on the Teach Pendant if needed. Always inform the team member(s) at the hardware station before commanding the robot. The arm moves to the detected pose in the final section. Keep the workspace clear.

1.1 Prerequisites

This part assumes you have completed Part 1 and are comfortable with ROS 2 topics, message interfaces, nodes, and actions. Familiarity with rigid-body frames and transforms (Lab 1) is also assumed.

1.2 Getting this Lab's Code

The hardware procedure lives on the `hardware` branch of the lab-6 repository. From your host system, navigate to the `~/Labs` directory and clone that branch:

```
cd ~/Labs
git clone -b hardware https://github.com/ENEE467-F2026/lab-6.git lab-6-hw
cd lab-6-hw/docker
```

> Both parts of this lab run on ROS 2 Jazzy. This part adds the RealSense camera and the real UR3e arm. The UR driver, MoveIt, and RealSense packages all run on Jazzy. Because no gripper is used in this part, no change of distribution is needed.

Launch the Docker container:

```
userid=$(id -u) groupid=$(id -g) docker compose -f lab-6-compose.yml run \
--rm lab-6-docker
```

You should be greeted with the container prompt `(lab-6) robot@docker-desktop: $`. Then build and source your workspace:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

1.3 Hardware Setup and Verification

Verify the two hardware components used in this part: the RealSense camera and the UR3e arm. The Hand-E gripper is not used here.

A. **RealSense D435i RGB-D Camera**: open the viewer and confirm the USB standard is at least 3.0:

```
realsense-viewer
```

Close the viewer, then confirm the camera streams in ROS 2:

```
ros2 launch realsense2_camera rs_pointcloud_launch.py
```

An RViz window with color, depth, and point-cloud streams confirms the camera is configured. Terminate the camera node and proceed.

B. **UR3e Robot Arm**: power on the Teach Pendant (TP), enable the robot to the `IDLE` state, and confirm it is reachable:

```
ping 192.168.77.22
```

C. **Workspace Props**: place a single rigid object (for example, a box or cup) on the table within the camera's field of view.

2. Point Clouds in ROS 2

A depth camera reports the scene as a *point cloud*: an unordered set of 3D points, each carrying a position and, for an RGB-D camera, a color. ROS 2 carries point clouds in the `sensor_msgs/PointCloud2` message. The RealSense driver publishes the aligned, colored cloud on:

```
/camera/camera/depth/color/points
```

Bring up the camera (if it is not already running) and inspect a single message:

```
ros2 topic echo --once /camera/camera/depth/color/points
```

The message header carries a `frame_id` (the camera's optical frame) and the body carries a packed array of points described by the `fields` entries (`x`, `y`, `z`, and a packed `rgb`). Because the points are expressed in the camera optical frame, every downstream stage that reasons about the robot's workspace must first transform them into the robot `base_link` frame.

Question 1. Inspect the echoed message. What is the `frame_id` of the published cloud, and what does each `PointCloud2` field (`x`, `y`, `z`, `rgb`) represent? Why must the cloud be transformed into `base_link` before the robot can act on a detected pose?

3. Point-Cloud Filtering: Voxel Downsampling and Cropping

A raw RealSense cloud contains hundreds of thousands of points per frame, most of them irrelevant to the task. The first stage reduces the cloud with a *voxel-grid* filter and crops it to the workspace. The `pc_voxel_filter_node` subscribes to the raw cloud and republishes a downsampled, cropped cloud on `/filtered_cloud`. Its key parameters are:

- `input_topic` (`/camera/camera/depth/color/points`): the raw cloud.
- `output_topic` (`/filtered_cloud`): the filtered cloud.

- `leaf_size` (0.005 m): the side length of each voxel cell. One representative point is kept per occupied cell, so a larger leaf size yields fewer points.
- `crop_bounds` ($[x_{\min}, x_{\max}, y_{\min}, y_{\max}, z_{\min}, z_{\max}]$): an axis-aligned box, in the working frame, outside of which points are discarded.

Bring up the full perception pipeline (camera, filter, segmentation, and pose server) with the provided launch file:

```
ros2 launch ur3e_hande_perception perception.launch.py
```

Then confirm the filtered cloud is being published and is much smaller than the raw cloud:

```
ros2 topic echo --once /filtered_cloud
```

Question 2. Increasing `leaf_size` reduces the number of points in `/filtered_cloud`. State one benefit and one cost of using a larger leaf size for the downstream plane segmentation and clustering stages.

4. Plane Segmentation and Object Clustering

The filtered cloud still contains both the tabletop and the objects on it. The `pc_segmentation_node` separates them in two steps, working in the frame set by the `base_frame` launch argument (default `base_link`):

Expressing poses in `base_link` requires a transform from `base_link` to the camera frame. The launch ships a placeholder static transform. On real hardware, replace it with your measured camera-to-base calibration so that detected poses land where the object actually is.

1. **RANSAC plane segmentation**: fit the dominant plane (the tabletop) by random sample consensus. Points within `plane_distance_thresh` (0.01 m) of the fitted plane are labeled as surface. The rest are candidate object points.
2. **DBSCAN clustering**: group the non-plane points into objects by density. Two parameters govern the clustering: `dbscan_eps` (0.03 m), the neighborhood radius, and `dbscan_min_samples` (50), the minimum number of neighbors for a core point.

The node publishes the fitted plane on `/plane_marker`, the object clusters on `/object_markers`, and per-object metadata on `/object_metadata`. With the pipeline running, the RViz window shows the filtered cloud, a plane marker for the table, and a bounding box around the detected object. Echo the detection summary:

```
ros2 topic echo --once /object_detected
```

Question 3. RANSAC and DBSCAN play complementary roles here. In one or two sentences each, explain (a) why RANSAC is well suited to extracting the tabletop plane from a cloud that also contains objects, and (b) why DBSCAN, rather than a fixed number of clusters, is

appropriate for grouping the remaining points into objects.

5. Serving Object Poses via an Action

Echoing a topic is convenient at the command line but does not let another node *request* a specific object on demand. The `obj_pose_action_server` wraps the detections in a ROS 2 action, `ur3e_hande_planning_interfaces/action/GetTargetObjPose`, served on `/get_target_obj_pose`. The goal describes the object the client wants, and the result returns its pose:

- **Goal**: `target_position` (an approximate centroid in `base_link`), `target_height`, and an optional `target_obj_bounds` box.
- **Result**: `target_obj_pose` (a `PoseStamped`) and `target_obj_found` (1 if a match was found, 0 otherwise).

With the pipeline running, request the object's pose (replace the position with the values you read from `/object_detected`):

```
ros2 action send_goal /get_target_obj_pose \
ur3e_hande_planning_interfaces/action/GetTargetObjPose \
"{target_position: {x: 0.28, y: 0.50, z: 0.05}, \
target_height: 0.12, target_obj_bounds: [0.20, 0.33, 0.0, 0.0]}"
```

The server reports success and returns the matched object's pose.

Question 4. Why is an action, rather than a service, the appropriate interface for serving object poses to a motion-planning client? In particular, what could go wrong in the move-to-pose task if the client blocked on a service call while the object detection was still settling?

6. Moving the Arm to the Object Pose

With a pose in hand, the final stage commands the UR3e to move its tool to the detected object's pose. This uses MoveIt for the arm only. No gripper is involved. Start the UR3e driver and MoveIt as in Lab 5 Part 3 (TP in `Play` mode), then run the move-to-pose client from the `ur3e_hande_moveit_py` package:

```
ros2 run ur3e_hande_moveit_py move_to_obj_pose
```

The client requests the object pose through `/get_target_obj_pose`, applies a small vertical standoff so the tool stops above the object rather than colliding with it, and plans and executes an arm motion to that standoff pose with MoveIt.

Question 5. The client moves the tool to a pose offset above the detected object rather than to the object pose itself. Give two reasons this standoff is used, given that the arm carries no gripper in this part.

7. Exercises

Exercise 1. Tuning the Perception Pipeline (35 pts.)

The default pipeline parameters are tuned for one table layout and may detect multiple objects or none in your setup. Your task is to make the pipeline reliably detect the single object on your table.

Deliverables: Complete this exercise by performing the following steps:

a. With the pipeline running, observe the RViz markers. If no object or more than one object is detected, adjust the `crop_bounds` of `pc_voxel_filter_node` (in `perception.launch.py`) so that only the object and the table beneath it remain in `/filtered_cloud`. Report your final `crop_bounds`.

b. If the object is split into several clusters or merged with the table edge, adjust `dbscan_eps` and `dbscan_min_samples` until exactly one object cluster is reported on `/object_detected`. Report your final values and explain the effect of each.

c. Include a screenshot of the RViz window showing the filtered cloud, the plane marker, and a single object bounding box.

Exercise 2. Move-to-Pose Client (35 pts.)

In this exercise you complete the move-to-pose client in `move_to_obj_pose.py`. The node already sends the action goal and receives the object pose. Your task is to turn that pose into a safe arm target.

Deliverables:

a. In the `TODO (Exercise 2a)` block, set the target z to the detected object's z plus the 0.10 m `STANDOFF_Z` so the tool stops above the object. Submit the code snippet.

b. Add a guard so that if `target_obj_found` is 0, the node logs an error and exits without planning. Submit the code snippet.

c. With the arm clear to move, run the client and confirm the tool reaches the standoff pose above the object. In one or two sentences, describe how the executed tool position compared with the pose reported by the action server.

8. Conclusion and Lab Report Instructions

After completing the lab, summarize the procedure and results into a well written lab report. Include your solutions to all of the question boxes in the lab report. Refer to the List of Questions to ensure you don't miss any. Include your full solution to the exercises in your lab report. If you wrote any Python programs during the exercises, include these files as a separate attachment, appending your team's number and last names (in alphabetical order) to the file, e.g., **Jane Doe**, **Alice Smith**, and **Richard Roe** in **Team 90** submitting their modified `main.py` would submit the file `main_Team90_Doe_Roe_Smith.py` and the report `Lab_<LabNumber>_Report_Team90_Doe_Roe_Smith.pdf`, where `<LabNumber>` is the number for that week's lab, e.g., **0**, **1**, **2**, etc. Submit your lab report along with any code to ELMS under the assignment for that week's lab **by the deadline stated on the ELMS assignment**. See ELMS for a lab report template named `Lab_Report_Template.pdf`. The grading rubric is as follows:

Component	Points
Lab Report and Formatting	10
Question 1.	4
Question 2.	4
Question 3.	4
Question 4.	4
Question 5.	4
Exercise 1.	35
Exercise 2.	35
Total	100